

Software & Machine Learning Taskphase Report

Tasks 1–5: Version Control, Python Programming, CSV Processing, Data Cleaning & Visualization

Name: Dhruvam Wason

Registration Number: 251090051680

Domain: Software / ML

GitHub Repository Link: https://github.com/DhruvamW/Synergy_TP

Table of Contents

Table of Contents.....	2
Abstract.....	3
1. Introduction.....	3
2. Development Environment and Version Control	4
Repository Structure and Commits.....	4
Branches.....	4
Why Version Control Matters.....	5
Python Virtual Environments and Dependency Isolation.....	5
requirements.txt and .gitignore	5
3. Python and File Handling Concepts	5
Functions and Modular Programming	5
Lists and Dictionaries	5
File Reading, Writing, and Exception Handling.....	6
Type Hints and Command-Line Arguments.....	6
4. CSV Parsing and Pandas.....	6
Manual CSV Parsing.....	7
Pandas DataFrames	8
Type Conversion and Summary Generation.....	8
Manual Parsing vs. Pandas: A Comparison.....	8
5. Data Cleaning.....	9
Common Data Quality Issues	9
Cleaning Decisions Made in Task 4	9
Why Data Cleaning Matters.....	10
6. Visualization	10
Why Labels and Titles Matter.....	10
Bar Charts vs. Scatter Plots.....	10
Average Score by Domain	11
Attendance vs. Score.....	11
Submission Status Count	12
7. Conclusion	12
References.....	13

Abstract

This report summarizes the work done in Tasks 1-5 of the Software and Machine Learning Taskphase, which is a gradual series of tasks aimed to develop real software engineering and data-processing skills. The challenges progressed from basic concepts like version control and Python programming to more difficult issues like CSV file processing, data cleaning, validation, and visualization. Throughout the taskphase, good version control habits were built using Git and GitHub, essential Python concepts like functions, file handling, and exception handling were used, and data manipulation was done with both manual parsing approaches and the pandas module. The subsequent tasks were on identifying and fixing data quality issues, followed by the production of visualizations to clearly communicate findings.

1. Introduction

The progression began with the fundamentals of software development practice — setting up a development environment, using version control, and writing clean, well-organized Python code. From there, the taskphase moved into the domain of data processing, where the focus shifted from writing general-purpose programs to working with structured datasets in the form of CSV files. The final stages introduced the practical realities of working with real data: handling missing or inconsistent values, validating data quality, and presenting results visually so that they could be interpreted easily by others.

A recurring theme throughout the taskphase was the importance of modular programming and clean software practices. Breaking logic into small, well-defined functions, separating concerns clearly, and maintaining a clean project structure made the code easier to test, debug, and extend as requirements grew more complex. These habits mirror practices expected in professional and research software development, including in machine learning workflows, where data pipelines must be both correct and maintainable.

In brief, the five tasks covered the following areas:

- Task 1: Setting up a development environment and version control workflow using Git and GitHub.
- Task 2: Core Python programming concepts, including functions, data structures, and file handling.
- Task 3: Reading and processing CSV files, comparing manual parsing with the pandas library.
- Task 4: Cleaning a real dataset by addressing missing values, duplicates, and inconsistent entries.
- Task 5: Visualizing the cleaned dataset to communicate patterns and trends clearly.

Task	Objective	Main Concepts Learned
Task 1	Development Environment Setup	Git, GitHub, Virtual Environments, Repository Structure
Task 2	Python Fundamentals	Functions, Lists, Dictionaries, File Handling, Exceptions
Task 3	CSV Processing	Manual CSV Parsing, File I/O, Data Summarization
Task 4	Data Cleaning	Pandas, Missing Values, Validation, Type Conversion
Task 5	Data Visualization	Matplotlib, Bar Charts, Scatter Plots, Data Interpretation

2. Development Environment and Version Control

A reliable development environment and disciplined version control practice formed the foundation for all subsequent tasks. Git, a distributed version control system, was used to track changes to the project's source code over time, while GitHub served as the remote platform for hosting the repository, making the project visible, shareable, and recoverable from any machine.

Repository Structure and Commits

The repository was organized so that related files were grouped logically, separating source code, data files, and documentation. This structure makes it easier for anyone reviewing the project, including the participant's future self, to locate relevant files quickly. Commits were used to record incremental, meaningful changes to the codebase, each accompanied by a short descriptive message. Frequent, small commits create a clear history of how the project evolved and make it possible to isolate exactly when and why a particular change was introduced, which is invaluable when diagnosing bugs.

Branches

Branches allow new features or experimental changes to be developed in isolation from the main, stable version of the code. This means that work in progress does not interfere with a working version of the project, and changes can be reviewed or tested before being merged back in. Even in a single-contributor project, using branches builds the habit of separating exploratory work from finished, validated work.

Why Version Control Matters

Version control is important because it provides a safety net: changes can always be reverted, the history of a project is preserved, and multiple lines of development can be managed without overwriting one another. In collaborative or larger projects, it also enables several people to work on the same codebase simultaneously without conflict, which is essential to most modern software and data science teams.

Python Virtual Environments and Dependency Isolation

A Python virtual environment creates an isolated installation of Python and its packages for a single project, separate from the system-wide Python installation. This isolation prevents conflicts between projects that may require different, sometimes incompatible, versions of the same library. Without virtual environments, installing a package for one project could silently break another.

`requirements.txt` and `.gitignore`

A `requirements.txt` file lists the specific packages (and frequently their versions) on which a project is dependent, allowing anyone who clones the repository to recreate the identical environment with a single command. A `.gitignore` file specifies which files and directories, such as virtual environment folders, cache files, or local configurations, should be excluded from Git tracking in order to keep the repository clean and prevent big or sensitive files from being accidentally included. Collectively, these practices—well-organized repositories, detailed commits, branching, separated environments, and unambiguous dependency lists—improve cooperation and repeatability. They ensure that a project can be understood, rebuilt, and developed by someone other than its original inventor, such as a teammate, reviewer, or the same developer returning to the project after a pause.

3. Python and File Handling Concepts

The early tasks in the phase reinforced core Python programming concepts that underpin nearly all subsequent data-processing work.

Functions and Modular Programming

Functions allow a piece of logic to be written once and reused wherever it is needed, rather than being duplicated throughout a program. Organizing a program into small, purpose-specific functions — each responsible for one clear task — makes code easier to read, test in isolation, and modify without unintended side effects elsewhere. This modular approach proved especially valuable once the tasks grew to involve multiple processing steps, such as reading a file, validating its contents, and summarizing the result, since each step could be developed and debugged independently.

Lists and Dictionaries

Lists were used to store ordered collections of data, such as rows read from a file, while dictionaries were used to represent structured records, mapping descriptive keys to their corresponding values. This combination — lists of dictionaries — closely mirrors how tabular data is naturally represented in code, with each dictionary acting as one row and each key acting as a column name.

File Reading, Writing, and Exception Handling

Reading from and writing to files allowed programs to work with data that persists beyond a single run, rather than relying on data hard-coded into the program itself. Because file operations can fail for many reasons — a missing file, an unexpected format, or a permissions issue — exception handling was used to detect these failure cases and respond to them gracefully, rather than allowing the program to crash unexpectedly. This is particularly important when a program will later be run on data it has not seen before.

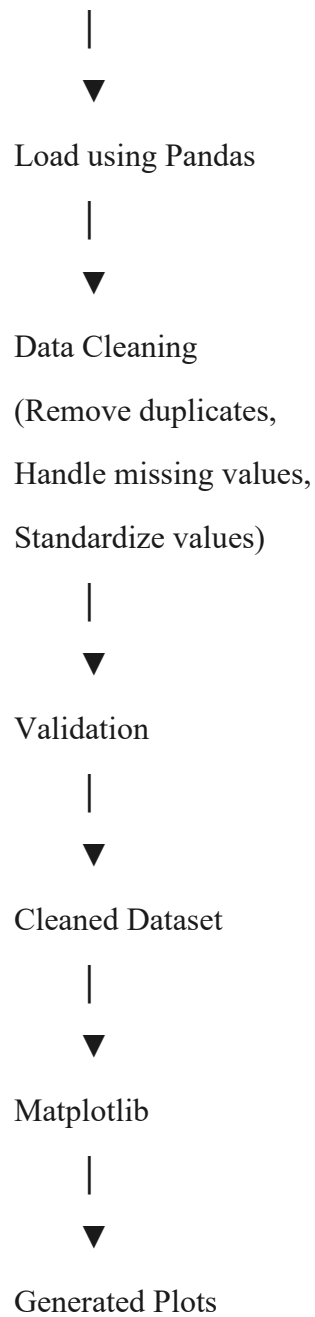
Type Hints and Command-Line Arguments

Type hints were used to indicate the expected type of a function's inputs and outputs, making code easier to understand and reducing the chance of passing the wrong kind of data into a function. Command-line arguments allowed scripts to be run with different inputs — such as a different file name — without needing to edit the source code itself, making the programs more flexible and reusable as standalone tools.

4. CSV Parsing and Pandas

CSV (comma-separated values) is a plain-text format for storing tabular data, where each line represents one row and values within a row are separated by a delimiter, most commonly a comma. The first row typically contains headers, naming each column, while every subsequent row contains the corresponding values for one record.

Raw CSV Dataset



Manual CSV Parsing

Parsing a CSV file manually involves opening the file, reading it line by line, splitting each line on the delimiter, and mapping the resulting values to their column headers. While straightforward for small, well-formed files, manual parsing requires explicit handling of edge cases such as values that contain the delimiter itself, inconsistent spacing, or missing fields, and offers no built-in support for type conversion or summarization.

Pandas DataFrames

Pandas is a Python library built specifically for working with tabular data. It represents a dataset as a DataFrame, a table-like structure with labeled rows and columns that supports fast, expressive operations such as filtering, grouping, and aggregating. Reading a CSV file into a DataFrame handles delimiter parsing, header detection, and basic type inference automatically, removing most of the manual bookkeeping required by hand-written parsing.

Type Conversion and Summary Generation

Once loaded, columns could be explicitly converted to appropriate types — for example, ensuring a score column is numeric rather than text — enabling correct mathematical operations. Pandas also provides built-in methods for generating summaries, such as column-wise averages, counts, and group-based statistics, which would otherwise require writing custom aggregation logic by hand.

Manual Parsing vs. Pandas: A Comparison

Reading data	Requires explicit file opening, line splitting, and delimiter handling	Single function call (<code>read_csv</code>) handles parsing internally
Type handling	All values read as strings; conversion must be coded manually	Automatic type inference for numeric and date columns
Missing data	Must be detected and handled with custom logic	Built-in detection and handling functions
Summarization	Aggregations (mean, count, group-wise stats) written from scratch	Built-in methods such as <code>describe()</code> , <code>groupby()</code> , <code>mean()</code>
Code volume & risk of error	Higher; more prone to edge-case bugs (extra spaces, quoted fields)	Lower; tested library code handles edge cases
Best suited for	Learning the underlying mechanics of file I/O and parsing	Practical, large-scale, real-world data analysis

In practice, manual parsing was valuable for understanding exactly how CSV data is structured and processed at a low level, while pandas proved far more efficient and reliable for the actual analysis work carried out in later tasks.

5. Data Cleaning

Real-world datasets are rarely perfectly structured, and Task 4 focused on identifying and resolving the kinds of imperfections commonly found in collected data before any meaningful analysis can take place.

Common Data Quality Issues

- Missing values: fields left blank or recorded as placeholders where no data was captured.
- Duplicate records: the same entry appearing more than once, which can distort summaries and counts.
- Invalid values: entries that fall outside a sensible or expected range (for example, a negative attendance count).
- Inconsistent categories: the same category represented in multiple ways (for example, differing capitalization or spelling of a label).
- Inconsistent units: values recorded under different units or scales that must be standardized before comparison.
- Incorrect data types: numeric or date values stored as plain text, preventing correct calculations.

Cleaning Decisions Made in Task 4

During Task 4, each of these issues was addressed deliberately rather than discarded by default. Missing numeric values were handled using context-appropriate strategies, such as imputing a reasonable default or excluding the record where imputation would be misleading. Exact duplicate rows were identified and removed to avoid double-counting. Category labels that referred to the same underlying value but were written inconsistently were standardized to a single consistent form. Values that fell outside plausible bounds were flagged and corrected or removed, and columns were explicitly converted to their correct data types to enable accurate downstream calculations.

Data Issue	Cleaning Rule
Duplicate Rows	Removed duplicate records
Missing Numeric Values	Replaced using the median

Invalid Attendance	Treated as missing before imputation
Domain Names	Standardized to consistent categories
Heights	Converted to centimetres
Weights	Converted to kilograms
Submitted Values	Standardized to "Yes" and "No"

Why Data Cleaning Matters

Validation and cleaning are necessary because the accuracy of any analysis, statistic, or visualization is entirely dependent on the quality of the underlying data. Uncleaned data can silently produce misleading averages, incorrect counts, or charts that misrepresent real patterns. Cleaning the dataset before analysis ensures that later results — including the visualizations produced in Task 5 — reflect genuine patterns in the data rather than artifacts of poor data entry or formatting inconsistencies.

6. Visualization

Visualization translates rows and columns of numbers into a form that can be interpreted quickly and intuitively. A well-chosen chart can reveal patterns, trends, or outliers that would be difficult to notice by scanning a table of raw values, making it an essential final step in any data analysis workflow.

Why Labels and Titles Matter

A chart without clear axis labels, units, or a descriptive title can be ambiguous or even misleading, since the viewer is forced to guess what is being measured. Clear labeling ensures that a visualization communicates the same information to any viewer, not just the person who created it, and is especially important when a chart will be shared or reviewed by others.

Saving visualizations as image files rather than displaying them only on screen makes the results reusable in reports, presentations, and documentation. It also allows the plots to be shared without rerunning the program, improving reproducibility and making the analysis easier to communicate.

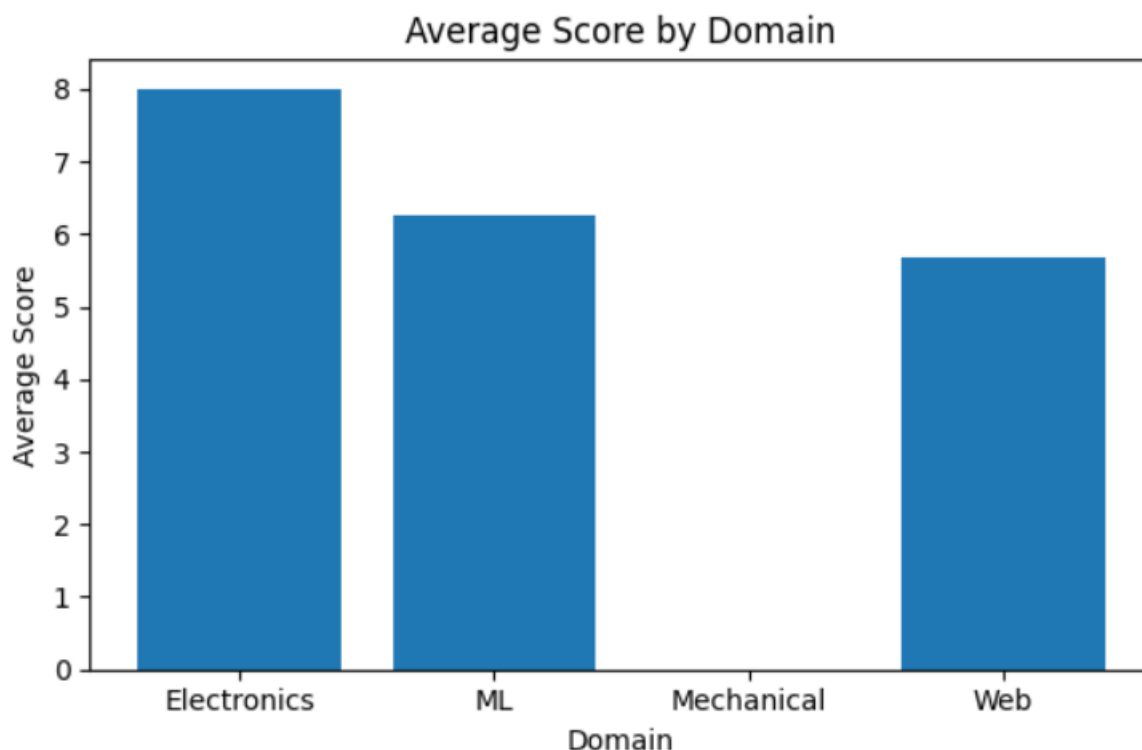
Bar Charts vs. Scatter Plots

Bar charts are best suited to comparing a quantity across distinct categories, where each bar's height or length represents a summary value such as an average or a count. Scatter plots, by

contrast, are used to examine the relationship between two continuous numeric variables, with each point representing one observation; they are particularly useful for spotting correlation, clustering, or outliers that a bar chart cannot show.

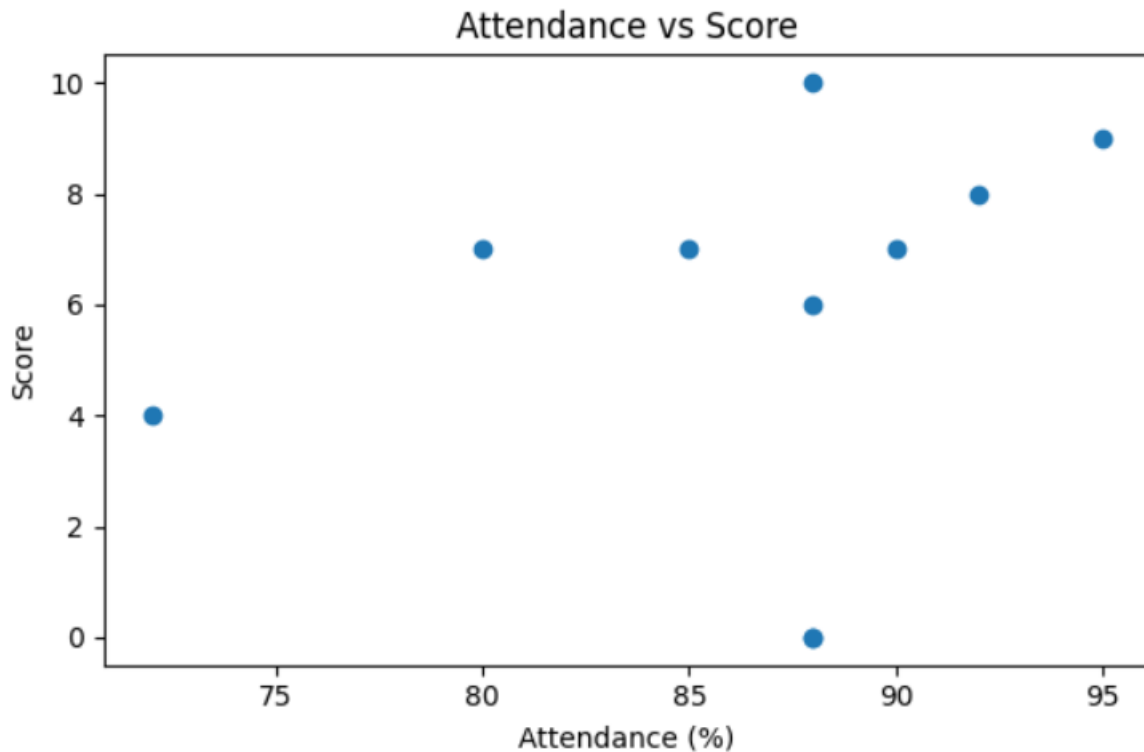
Average Score by Domain

This visualization used a bar chart to compare the average score achieved across different domains. Because domain is a categorical variable and average score is a single summary value per category, a bar chart made it straightforward to see which domains performed relatively higher or lower at a glance.



Attendance vs. Score

This visualization used a scatter plot to examine the relationship between attendance and score, with each point representing one participant. Plotting these two continuous variables against each other made it possible to assess whether higher attendance was generally associated with higher scores, and to identify any individual points that deviated from the overall trend.



Submission Status Count

This visualization used a bar chart to display the number of records falling into each submission status category, such as submitted, late, or missing. Because the underlying value here is simply a count per category, a bar chart provided a clear, immediate sense of how submissions were distributed across the possible statuses.

7. Conclusion

Across the five tasks of this phase, the work progressed from establishing sound development practices to performing a complete, small-scale data analysis workflow: setting up version control and a Python environment, writing modular and well-structured code, parsing and processing CSV data, cleaning a dataset to ensure its reliability, and visualizing the results to communicate findings clearly.

Beyond the specific technical concepts covered, the taskphase reinforced general programming skills, including writing more deliberate, organized, and reusable code, and a more careful, validation-first approach to working with data. The emphasis on clean data and clear visualization in particular underscored a principle that extends well beyond these five tasks: the value of any

analysis or model is limited by the quality of the data it is built on, and the value of any result is limited by how clearly it is communicated.

Taken together, these tasks built a practical foundation — in tooling, programming discipline, and data handling — that mirrors the early stages of any applied machine learning workflow, where reliable code, clean data, and clear communication of results are prerequisites for the more advanced modelling work to follow.

References

- Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
- Pandas Development Team. Pandas Documentation. <https://pandas.pydata.org/docs/>
- Matplotlib Development Team. Matplotlib Documentation.
<https://matplotlib.org/stable/contents.html>
- <https://www.geeksforgeeks.org/pandas/pandas-tutorial/>